

Assignment-3

(Advance Python Programming)

[Printing on Screen]

1. Introduction to the print() function in Python.

The print() function is one of the most commonly used functions in Python. It is used to display output on the screen (console). When a Python program runs, the print() function helps programmers view messages, results, and values generated by the program.

The print() function is often the first function beginners learn because it allows them to interact with the user by showing information in a readable format.

➤ **Features of print()**

- Displays text, numbers, and other data types on the screen.
- Helps in checking program output and debugging.
- Can print multiple values at the same time.
- Automatically converts many data types into a readable format before displaying them.

➤ **Importance of print()**

- Used for communication between the program and the user.
- Helps programmers understand how a program is working.
- Useful for testing and debugging code during development.
- Makes program output clear and organized.

2. Formatting outputs using f-strings and format().

➤ **f-strings**

f-strings (formatted string literals) were introduced in Python 3.6. They provide a simple and efficient way to embed variables and expressions directly inside strings.

Features:

- Easy to read and write.
- Faster than many other formatting methods.
- Allows expressions and calculations inside strings.

➤ **format() Method**

The format() method is used to insert values into a string by replacing placeholders with specified data.

Features:

- Works in older Python versions.
- Supports flexible formatting options.
- Useful for creating structured and customized output.

[Reading Data from Keyboard]

3. Using the input() function to read user input from the keyboard.

The input() function in Python is used to accept data entered by the user through the keyboard. It allows a program to interact with users by receiving information during execution. When the input() function is executed, the program pauses and waits for the user to enter a value. After the user presses Enter, the entered data is returned as a string and can be used within the program.

➤ Features of input()

- Accepts user input from the keyboard.
- Enables interaction between the user and the program.
- Returns the entered value as a string.
- Can be used to collect different types of information such as names, numbers, and messages.

4. Converting user input into different data types (e.g., int, float, etc.).

The input() function always returns data as a string. When numerical operations are required, the user input must be converted into the appropriate data type, such as int, float, or others. This process is known as type conversion or type casting.

Common Data Type Conversions:

- int() – Converts input into an integer (whole number).
- float() – Converts input into a floating-point number (decimal value).
- str() – Converts data into a string.
- bool() – Converts data into a Boolean value (True or False).

Importance of Type Conversion:

- Allows mathematical calculations on user-entered numbers.
- Ensures data is processed in the correct format.
- Helps prevent errors caused by incorrect data types.
- Makes programs more flexible and user-friendly.

[Opening and Closing Files]

5. Opening files in different modes ('r', 'w', 'a', 'r+', 'w+').

Python uses different file modes to determine how a file will be accessed and manipulated. The mode is specified when opening a file and defines whether the file will be read, written, or both.

➤ 'r' (Read Mode)

- Opens a file for reading only.
- The file must already exist.
- No changes can be made to the file.

➤ 'w' (Write Mode)

- Opens a file for writing only.

- Creates a new file if it does not exist.
- If the file already exists, its contents are overwritten.
- 'a' (Append Mode)
 - Opens a file for writing.
 - Creates a new file if it does not exist.
 - New data is added to the end of the file without removing existing content.
- 'r+' (Read and Write Mode)
 - Opens a file for both reading and writing.
 - The file must already exist.
 - Allows modification of existing content.
- 'w+' (Write and Read Mode)
 - Opens a file for both writing and reading.
 - Creates a new file if it does not exist.
 - If the file already exists, its contents are erased before writing.

6. Using the open() function to create and access files.

The open() function is used in Python to open, create, and access files. It establishes a connection between the program and a file, allowing data to be read from or written to the file. When a file is opened, Python returns a file object that can be used to perform various file operations. The behavior of the file depends on the mode specified while opening it.

- **Features of open()**
 - Opens existing files for reading or writing.
 - Creates a new file if it does not exist (depending on the mode used).
 - Supports different file modes such as read, write, and append.
 - Enables file handling operations like reading, writing, and updating data.
- **Importance of open()**
 - Allows permanent storage of data in files.
 - Helps manage and retrieve information efficiently.
 - Essential for applications that work with external data.
 - Provides a way to interact with text and other file types.

7. Closing files using close().

The close() method is used to close an opened file in Python. After all file operations are completed, the file should be closed to release system resources and ensure that all data is properly saved.

Closing a file breaks the connection between the program and the file, preventing further read or write operations unless the file is opened again.

- **Features of close()**
 - Closes an opened file.
 - Frees memory and system resources.

- Ensures that all written data is saved correctly.
 - Prevents accidental modifications after file operations are completed.
- **Importance of close()**
- Improves program efficiency by releasing resources.
 - Helps avoid data loss or corruption.
 - Ensures proper file management.
 - Considered a good programming practice.

[Reading and Writing Files]

8. Reading from a file using read(), readline(), readlines().

- **read()**
- Reads the entire content of a file at once.
 - Returns the data as a single string.
 - Useful for small files.
- **readline()**
- Reads one line from a file at a time.
 - Returns the line as a string.
 - Useful when processing files line by line.
- **readlines()**
- Reads all lines of a file and stores them in a list.
 - Each line becomes an element in the list.
 - Useful when individual lines need to be accessed separately.

9. Writing to a file using write() and writelines().

- **write()**
- Writes a single string to a file.
 - Used to store text or data in a file.
 - Does not automatically add a new line.
- **writelines()**
- Writes multiple lines or a collection of strings to a file.
 - Useful for writing lists of text.
 - Does not automatically insert newline characters.

[Exception Handling]

10. Introduction to exceptions and how to handle them using try, except, and finally.

An exception is an error that occurs during the execution of a program. When an exception occurs, the normal flow of the program is interrupted. Python provides exception handling mechanisms to manage such errors and prevent program crashes.

- try Block
 - Contains the code that may cause an exception.
 - Python checks this block for errors during execution.
- except Block
 - Executes when an exception occurs in the try block.
 - Handles the error and prevents the program from terminating unexpectedly.
- finally Block
 - Always executes whether an exception occurs or not.
 - Commonly used for cleanup tasks such as closing files or releasing resources.

11. Understanding multiple exceptions and custom exceptions

A program may encounter different types of exceptions. Python allows multiple except blocks to handle different exceptions separately. This helps provide specific error messages and appropriate handling for each type of error.

- **Benefits of Multiple Exceptions**
 - Handles different errors individually.
 - Makes programs more reliable and easier to debug.
 - Improves error reporting.

- **Understanding Custom Exceptions**

Python allows programmers to create their own exceptions, known as **custom exceptions**. These are useful when built-in exceptions do not adequately represent a specific error condition in an application.

- **Benefits of Custom Exceptions**
 - Provides meaningful error messages.
 - Makes code easier to understand and maintain.
 - Helps handle application-specific errors effectively.

[Class and Object (OOP Concepts)]

12. Understanding the concepts of classes, objects, attributes, and methods in Python.

➤ Class

A **class** is a blueprint or template used to create objects. It defines the properties (attributes) and behaviors (methods) that objects of that class will have.

➤ Object

An **object** is an instance of a class. It represents a real-world entity and contains its own data and functionality based on the class definition.

➤ Attribute

An **attribute** is a variable associated with a class or object. Attributes store information or data about an object.

➤ Method

A **method** is a function defined inside a class. It specifies the actions or behaviors that an object can perform.

➤ Importance of Classes and Objects

- Helps organize code efficiently.
- Supports code reusability.
- Makes programs easier to maintain and understand.
- Forms the foundation of Object-Oriented Programming (OOP).

13. Difference between local and global variables.

➤ Local Variable

- Declared inside a function or method.
- Accessible only within that function or method.
- Exists only while the function is executing.

➤ Global Variable

- Declared outside all functions.
- Accessible throughout the program.
- Remains available during the entire program execution.

Local Variable	Global Variable
Defined inside a function	Defined outside functions
Accessible only within the function	Accessible throughout the program
Temporary in nature	Exists for the program's lifetime
Used for specific tasks	Used for shared data

[Inheritance]

14. Single, Multilevel, Multiple, Hierarchical, and Hybrid inheritance in Python

Inheritance is an Object-Oriented Programming (OOP) feature that allows one class to acquire the properties and methods of another class. It promotes code reusability and reduces redundancy.

➤ **Single Inheritance**

- A child class inherits from one parent class.
- Simplest form of inheritance.

➤ **Multilevel Inheritance**

- A class inherits from another child class.
- Forms a chain of inheritance across multiple levels.

➤ **Multiple Inheritance**

- A child class inherits from more than one parent class.
- Allows access to features from multiple classes.

➤ **Benefits of Inheritance**

- Promotes code reusability.
- Reduces duplication of code.
- Improves maintainability.
- Supports hierarchical classification of classes.

15. Using the `super()` function to access properties of the parent class.

The `super()` function is used to access methods and properties of a parent class from a child class. It helps in reusing the functionality of the parent class without directly referring to its name.

➤ **Features of `super()`**

- Calls parent class methods from a child class.
- Supports inheritance and code reuse.
- Makes code easier to maintain.
- Useful when working with multiple levels of inheritance.

➤ **Advantages of `super()`**

- Avoids hardcoding parent class names.
- Improves code readability.
- Simplifies method overriding.
- Supports cooperative inheritance in complex class hierarchies.

[Method Overloading and Overriding]

16.Method overloading: defining multiple methods with the same name but different parameters.

Method Overloading is the concept of defining multiple methods with the same name but different parameters. In many programming languages, method overloading allows a method to perform different tasks based on the number or type of arguments passed.

In Python, true method overloading is not directly supported. However, similar behavior can be achieved using default arguments or variable-length arguments.

➤ **Benefits of Method Overloading**

- Improves code readability.
- Increases flexibility in method usage.
- Allows the same method name to handle different situations.

17.Method overriding: redefining a parent class method in the child class

Method Overriding occurs when a child class provides its own implementation of a method that is already defined in the parent class. The method in the child class replaces the parent class method when called through a child object.

➤ **Benefits of Method Overriding**

- Allows customization of inherited behavior.
- Supports runtime polymorphism.
- Improves flexibility and extensibility of programs.
- Enables child classes to provide specific implementations.

[SQLite3 and PyMySQL (Database Connectors)]

18.Introduction to SQLite3 and PyMySQL for database connectivity.

➤ **SQLite3**

SQLite3 is a lightweight, serverless, and self-contained relational database management system. Python provides built-in support for SQLite through the sqlite3 module, making it easy to create and manage databases without installing additional software.

➤ **Features of SQLite3**

- Lightweight and easy to use.
- No separate server required.
- Stores the database in a single file.
- Ideal for small to medium-sized applications.

➤ **PyMySQL**

PyMySQL is a Python library used to connect Python applications with a MySQL database server. It enables Python programs to perform database operations such as creating tables, inserting records, updating data, and retrieving information.

➤ **Features of PyMySQL**

- Connects Python with MySQL databases.
- Supports SQL query execution.
- Suitable for web and enterprise applications.
- Provides efficient database management capabilities.

19. Creating and executing SQL queries from Python using these connectors.

SQL (Structured Query Language) is used to communicate with databases. Python can execute SQL commands through database connectors such as SQLite3 and PyMySQL.

Common SQL Operations

➤ **CREATE**

- Used to create databases and tables.
- Defines the structure of data storage.

➤ **INSERT**

- Adds new records into a table.
- Stores user or application data.

➤ **SELECT**

- Retrieves data from a table.
- Used for searching and displaying records.

➤ **UPDATE**

- Modifies existing records in a table.
- Helps keep data current and accurate.

➤ **DELETE**

- Removes records from a table.
- Used when data is no longer needed.

Benefits of Executing SQL Queries from Python

- Automates database operations.
- Enables dynamic data management.
- Integrates databases with Python applications.
- Simplifies data storage and retrieval.

[Search and Match Functions]

20. Using `re.search()` and `re.match()` functions in Python's `re` module for pattern matching.

The `re` module in Python provides support for regular expressions (regex), which are used to search, match, and manipulate text based on specific patterns.

➤ **`re.search()`**

- Searches the entire string for a specified pattern.
- Returns the first match found anywhere in the string.
- Returns `None` if no match is found.

➤ **`re.match()`**

- Checks for a match only at the beginning of the string.
- Returns a match object if the pattern is found at the start.
- Returns `None` if the pattern does not appear at the beginning.

Importance of Pattern Matching

- Helps validate user input.
- Used for text processing and data extraction.
- Useful in searching, filtering, and analyzing text data.
- Simplifies handling of complex string patterns.

21. Difference between search and match

<code>re.search()</code>	<code>re.match()</code>
Searches the entire string for a pattern.	Checks only the beginning of the string.
Finds a match anywhere in the text.	Finds a match only if the pattern starts at the first character.
More flexible for general searching.	Useful when the pattern must appear at the start.
Returns the first occurrence of the pattern in the string.	Returns a match only from the beginning of the string.